

The Four Caches in LLM Serving

Four independent caches sit between a prompt and a token. They are owned by different parties, keyed on different things, and only one of them can change what the model says. This note derives each from first principles — the arithmetic, the latency it buys, the architecture that makes it correct, and the code that implements it.

WHO THIS IS FOR

Engineers who call or serve LLMs and want to reason about latency and cost from first principles, not folklore.

Assumes familiarity with a transformer layer and what attention computes. No serving-engine experience required.

THE BOTTOM LINE

85%

INPUT SPEND

less per 30-turn agent episode with a provider prompt cache — 879,000 billed tokens down to 131,600.

35×

TIME TO FIRST TOKEN

faster when 97% of a 21,200-token prompt is served from prefix-cached blocks: ~7.0 s to ~0.2 s.

1 of 4

LOSSY LAYERS

can return a wrong answer. Three reuse attention state and are lossless; the semantic cache matches on meaning.

WHAT YOU WILL BE ABLE TO DO

1. Estimate KV-cache memory and decode-step latency for any model from its config file.
2. Predict how much prefill a prefix cache will save for a given prompt layout, and lay prompts out to maximise it.
3. Compute break-even and total cost for provider prompt caching in a multi-turn agent.
4. Decide whether a semantic cache is worth deploying, and tune its threshold with a measured precision/recall curve.
5. Explain which of the four layers changes model output — and which cannot.

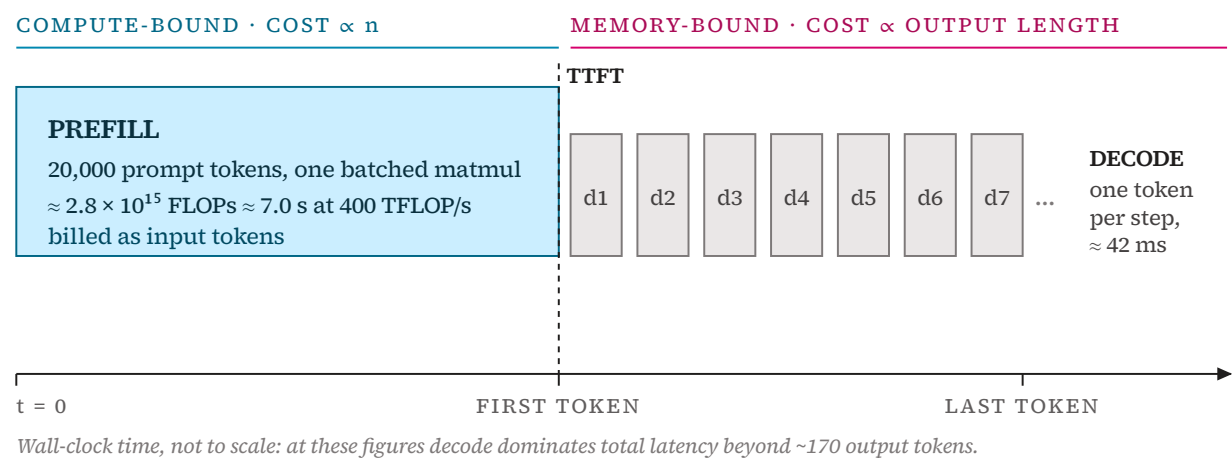
SECTION 0

The mental model: prefill versus decode

Serving a request has two phases with very different cost profiles. **Prefill** reads the whole prompt and, for every token at every layer, computes the key (K) and value (V) projections attention needs. It is a large, parallel matrix multiply: compute-bound, growing linearly with prompt length plus a quadratic attention term that matters at long context. **Decode** produces one token per step; each step is tiny in FLOPs but must stream all the weights — and the whole KV cache — from GPU memory, so it is limited by memory bandwidth, not arithmetic.

EXHIBIT 1

One compute-bound burst, then thousands of memory-bound steps



In an agent loop most prefill is redundant: each turn appends a few hundred tokens to a context that is otherwise unchanged. The four caches exist to avoid recomputing — or re-paying for — those tokens.

NOTATION

L	number of transformer layers
n_kv	key/value heads (= attention heads without GQA)
d_h	head dimension
d_model	hidden size
n	prompt length in tokens
P	parameter count
b	bytes per element (2 bf16/ fp16, 1 fp8)
BW	aggregate GPU memory bandwidth (bytes/s)
F	achievable throughput (FLOP/s)

BASELINE PREFILL COST

$$\text{FLOPs}_{\text{prefill}} \approx \underbrace{2Pn}_{\text{linear layers}} + \underbrace{4Ln^2d_{\text{model}}}_{\text{attention scores}}$$
$$\text{TTFT} \approx \text{FLOPs}_{\text{prefill}} / F$$

THE RUNNING EXAMPLE

A tool-using agent with a **20,000-token stable context** (system prompt, tool schemas, reference docs) that appends **Δ = 600** tokens per turn for **K = 30** turns. Served by a Llama-3-70B-class model (L=80, n_kv=8, d_h=128, P=70×10⁹, bf16) on H100-class hardware.

First term of the prefill formula: 2 · 70×10⁹ · 20,000 ≈ 2.8×10¹⁵ FLOPs — about **7 seconds** at 400 TFLOP/s realised bf16 throughput. That is the number each cache is trying to shrink.

On the hardware numbers. A 70B model in bf16 needs ~140 GB of weights and does not fit on one 80 GB GPU; in practice it is sharded across a node. Sharding divides both weight bytes and KV bytes across GPUs, so the *ratios* below hold; absolute figures use single-H100 bandwidth (3.35 TB/s) for simplicity. Treat them as order-of-magnitude.

LAYER 1

The KV cache — per request, on the GPU

Owner: the model runtime. Lifetime: one request. Saves: quadratic projection work in decode.

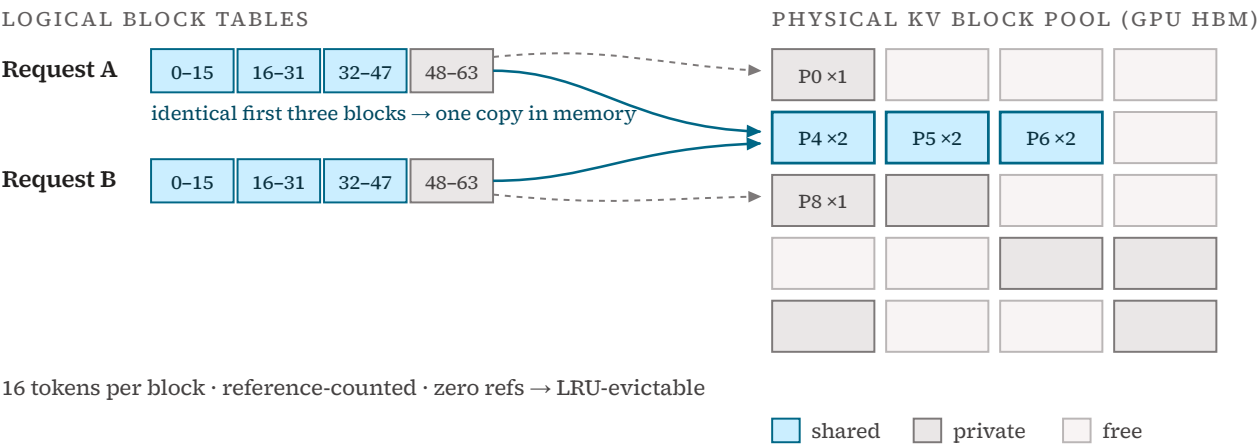
ARCHITECTURE

During decoding, each new token attends to every earlier token. Attention needs the **keys and values** of all previous positions; the **query** is needed only for the current position. That asymmetry is why K and V are cached and Q is not. The KV cache stores, for each layer and each KV head, one key vector and one value vector per token — tensors shaped `[batch, n_kv, seq_len, d_h]`. At each decode step the model computes K and V only for the newest token, appends them, and reads the rest from memory.

Production engines do not allocate this as one contiguous buffer. **PagedAttention** (vLLM) divides the sequence into fixed-size blocks addressed through a per-request block table — the same idea as virtual-memory paging. This bounds fragmentation and, crucially, lets physical blocks be shared between requests, which is what makes prefix caching possible.

EXHIBIT 2

Paged KV memory: logical block tables over a shared physical pool



16 tokens per block · reference-counted · zero refs → LRU-evictable

Blocks are physically interchangeable; the block table, not the allocator, decides what a sequence sees.

Paging bounds fragmentation and makes sharing expressible. Without it, a shared prefix would still have to be copied per request.

MATHS

$$\text{bytes per token} = \underbrace{2}_{K \text{ and } V} \cdot L \cdot n_{kv} \cdot d_h \cdot b$$

Running example: $2 \cdot 80 \cdot 8 \cdot 128 \cdot 2 = 327,680 \text{ B} \approx$
320 KB per token. The 20k-token agent context
occupies **6.4 GB**; a 128k context needs **42 GB** —
before the 140 GB of weights.

This is why grouped-query attention — cutting n_{kv} from 64 to 8 — was worth an 8× reduction in cache size, and why fp8 KV quantisation is attractive.

Compute saved. Without a cache, generating token t requires recomputing K/V for all $t-1$ previous tokens, so producing T output tokens costs $O(T^2)$ projection work. With the cache it is $O(T)$.

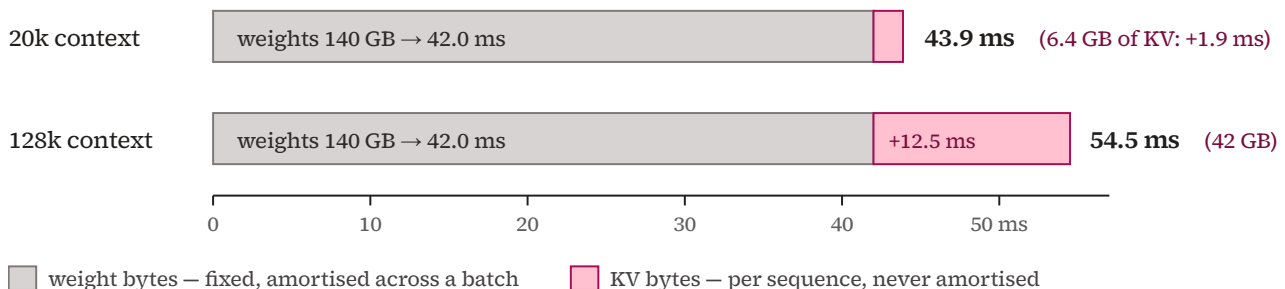
PERFORMANCE

Why is decode memory-bound? Arithmetic intensity for a single-token step is roughly 2 FLOPs per weight byte read — far below the ~300 FLOP/byte an H100 needs to be compute-limited. So step time is set by bytes streamed, not by arithmetic. Batching amortises the *weight* read across requests but not the *KV* read, which is per-request. Hence the operational conclusion: **KV footprint, not FLOPs, determines how many concurrent sequences a GPU can serve**.

EXHIBIT 3

Decode step time is bytes streamed ÷ bandwidth — and the KV share grows with context

$$t_{\text{step}} \approx (P \cdot b + n \cdot \text{bytes_per_token}) / \text{BW} \quad \cdot \quad \text{BW} = 3.35 \text{ TB/s}$$



+24% step time from reading the cache alone, at 6.4× the context. Nothing here helps the *next* turn — the KV cache is freed when the request ends.

Weight bytes are fixed and shareable across a batch; KV bytes are per sequence and scale with context. Long-context serving is a memory-bandwidth problem before it is a FLOPs problem.

CODE

A single-layer attention block with an explicit cache. On the prefill call `cache=None` and T is the prompt length; on each decode call $T=1$ and the cache carries everything before it. For clarity this uses one K/V head per query head; GQA would repeat n_{kv} K/V heads across query heads.

LISTING 1 · ATTENTION WITH AN EXPLICIT K/V CACHE

```

import torch
import torch.nn.functional as F

class CachedAttention(torch.nn.Module):
    def __init__(self, d_model: int, n_heads: int):
        super().__init__()
        self.h, self.d = n_heads, d_model // n_heads
        self.qkv = torch.nn.Linear(d_model, 3 * d_model, bias=False)
        self.out = torch.nn.Linear(d_model, d_model, bias=False)

    def forward(self, x, cache=None):
        # x: [B, T_new, d_model]
        B, T, _ = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        q, k, v = (t.view(B, T, self.h, self.d).transpose(1, 2) for t in (q, k, v))
        if cache is not None:
            # decode: append only new K/V
            # [B, h, T_old + T_new, d]
            k = torch.cat([cache["k"], k], dim=2)
            v = torch.cat([cache["v"], v], dim=2)
        new_cache = {"k": k, "v": v}
        # A causal mask is needed only when several new tokens arrive at once
        # (prefill, or chunked prefill). For single-token decode the query
        # may see everything.
        y = F.scaled_dot_product_attention(q, k, v, is_causal=(cache is None))
        return self.out(y.transpose(1, 2).reshape(B, T, -1)), new_cache

```

In Hugging Face transformers the same object is `DynamicCache`, returned as `past_key_values`. Prefill a shared context once and reuse it across queries.

LISTING 2 · PREFILL ONCE, ANSWER MANY

```

import copy, torch
from transformers import AutoTokenizer, AutoModelForCausalLM, DynamicCache

tok = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id, torch_dtype=torch.bfloat16, device_map="auto")

prefix_ids = tok(shared_context, return_tensors="pt").input_ids.to(model.device)
prefix_cache = DynamicCache()
with torch.no_grad():
    model(prefix_ids, past_key_values=prefix_cache, use_cache=True) # prefill once
for question in questions:
    ids = tok(shared_context + question, return_tensors="pt").input_ids.to(model.device)
    out = model.generate(ids, past_key_values=copy.deepcopy(prefix_cache),
                        max_new_tokens=200)
    print(tok.decode(out[0, ids.shape[1]:], skip_special_tokens=True))

```

`generate` mutates the cache it receives, so each query starts from a deep copy of the prefilled state; it compares cache length to prompt length and prefills only the tokens past the cache.

LAYER 2

Prefix caching — across requests, in the serving engine

Owner: the serving engine. Lifetime: until LRU eviction. Saves: prefill FLOPs and time-to-first-token.

ARCHITECTURE

If two prompts share an identical leading token sequence, the K/V tensors for that span are bit-identical, because causal attention means position i depends only on positions $\leq i$. Prefix caching retains those blocks after a request finishes and lets later requests map to them. vLLM's Automatic Prefix Caching — on by default in recent versions — works as follows.

- KV memory is divided into **blocks of 16 tokens** (configurable).
- Each full block is identified by a **chained hash**: $h_i = H(h_{i-1}, \text{tokens}_i)$. Because h_i includes its predecessor, a block can only match if every block before it matched. This is a correctness requirement, not an optimisation: the K/V values in block i depend on all earlier tokens.
- A global table maps hash $\rightarrow$ physical block. Blocks are reference-counted; a block with zero references is eligible for **LRU eviction**.
- On admission the scheduler walks the prompt's block hashes, stops at the **first miss**, marks matched blocks as shared, and schedules prefill only for the suffix.

SGLang's **RadixAttention** reaches the same goal with a radix tree keyed on token sequences, which makes it cheaper to find the longest shared prefix among many concurrent requests. TensorRT-LLM and llama.cpp have equivalent mechanisms.

EXHIBIT 4

The chain rule of prompt layout: one changed token invalidates everything after it

$$h_i = H(h_{i-1}, \text{tokens}_i) \quad \cdot \quad n_{\text{cached}} = B \cdot \lfloor m / B \rfloor, \quad B = 16$$



Rule 1 Stable content first, variable content last.

Rule 2 Align breakpoints to block boundaries — pad a system prompt to a multiple of 16 tokens.

A timestamp, request ID or user name at the top of the prompt changes block 1's hash and therefore every hash after it — a 100% miss on an otherwise perfectly reusable prompt.

MATHS

$$f = n_{\text{cached}} / n$$
$$\text{FLOPs}'_{\text{prefill}} \approx (1 - f) \text{FLOPs}_{\text{prefill}}$$
$$\text{TTFT}' \approx (1 - f) \text{TTFT} + t_{\text{lookup}}$$

A 1,000-token shared prefix yields $16 \cdot 62 = 992$ cached tokens; the last 8 are recomputed. t_{lookup} is $O(n / B)$ hash computations and dictionary probes — microseconds against seconds of prefill. With 40 GB spare KV memory the cache holds $\approx 7,600$ blocks $\approx 122\text{k}$ tokens of distinct prefixes.

CODE

LISTING 3 · THE CHAINED-HASH LOOKUP, REDUCED TO ESSENTIALS

```
import hashlib

BLOCK = 16
def block_hashes(token_ids: list[int]) -> list[bytes]:
    """Hash of block i depends on hash of block i-1 — a mid-prompt match is impossible."""
    hashes, prev = [], b"root"
    usable = len(token_ids) - len(token_ids) % BLOCK    # only full blocks are cacheable
    for i in range(0, usable, BLOCK):
        h = hashlib.sha256(prev + str(token_ids[i:i + BLOCK]).encode()).digest()
        hashes.append(h)
        prev = h
    return hashes

def cached_prefix_len(token_ids: list[int], table: dict[bytes, int]) -> int:
    """Walk block by block; stop at the first miss."""
    n = 0
    for h in block_hashes(token_ids):
        if h not in table:
            break
        n += BLOCK
    return n

# Demonstration: a one-token change at the front invalidates everything.
table = {h: i for i, h in enumerate(block_hashes(list(range(1000))))}
print(cached_prefix_len(list(range(1000)), table))    # 992
print(cached_prefix_len([9] + list(range(1, 1000)), table))    # 0
print(cached_prefix_len(list(range(990)) + [9] + list(range(991, 1000)), table))    # 976
```

PERFORMANCE

Turn 2 of the running example shares the first 20,600 tokens with turn 1, so $f = 20,592 / 21,200 \approx 0.97$. Prefill drops from ~ 7 s to ~ 0.2 s. Every later turn behaves the same, because each turn's prompt is the previous prompt plus 600 tokens.

What remains is decode cost, which prefix caching does nothing about — a point worth repeating to anyone who expects it to speed up long generations.

LISTING 4 · TURNING IT ON IN VLLM

```
from vllm import LLM, SamplingParams

llm = LLM(model=model_id, enable_prefix_caching=True)  # default-on in V1; explicit here
outputs = llm.generate([shared_context + q for q in questions],
                        SamplingParams(max_tokens=200))

for o in outputs:
    print(o.num_cached_tokens)  # 0 for the first request, ≈ len(shared prefix) after
```

The attribute name has moved between vLLM versions; check RequestOutput in the version you run.

LAYER 3

Prompt caching — on a provider's hardware, with a price sheet

Owner: the API provider. Lifetime: a short TTL. Saves: input cost toward 10% of list, plus TTFT.

ARCHITECTURE

Prompt caching is prefix caching operated by an API provider and exposed as a **billing construct**. You cannot see the KV blocks; you can see the invoice and the usage block in each response. In Anthropic's interface, cache boundaries are declared with `cache_control`: `{"type": "ephemeral"}` on a content block; the cached prefix is everything up to and including that block, in the order **tools** → **system** → **messages**. Entries have a short TTL (minutes by default, refreshed on each read; a longer-TTL tier exists at a higher write price), and each model has a **minimum cacheable length** on the order of 1–2k tokens — shorter prefixes are silently not cached. Matching is exact-prefix, so the ordering discipline of Layer 2 applies unchanged. Other providers cache automatically on exact prefixes with a discounted read rate and no explicit write charge: the mechanics are the same, the accounting differs.

MATHS

Let p be the base input price per token, w the write multiplier and r the read multiplier. Anthropic's published figures at the time of writing: $w = 1.25$, $r = 0.10$.

$$\begin{aligned}\text{cost}_{\text{no cache}} &= K(n_p + n_s)p \\ \text{cost}_{\text{cache}} &= (wn_p + n_s)p + (K - 1)(rn_p + \Delta)p\end{aligned}$$

Break-even on the prefix: $w + (K-1) \cdot r < K \Rightarrow K > (w - r)/(1 - r) \approx 1.28$. **One reuse within the TTL already pays for the write.** As $K \rightarrow \infty$ the prefix cost approaches 10% of baseline.

In a **multi-turn agent loop**, turn t appends Δ tokens with the breakpoint at the end of the latest message. Turn 1 writes the whole prompt; each later turn reads everything before the new tokens and writes only the new Δ .

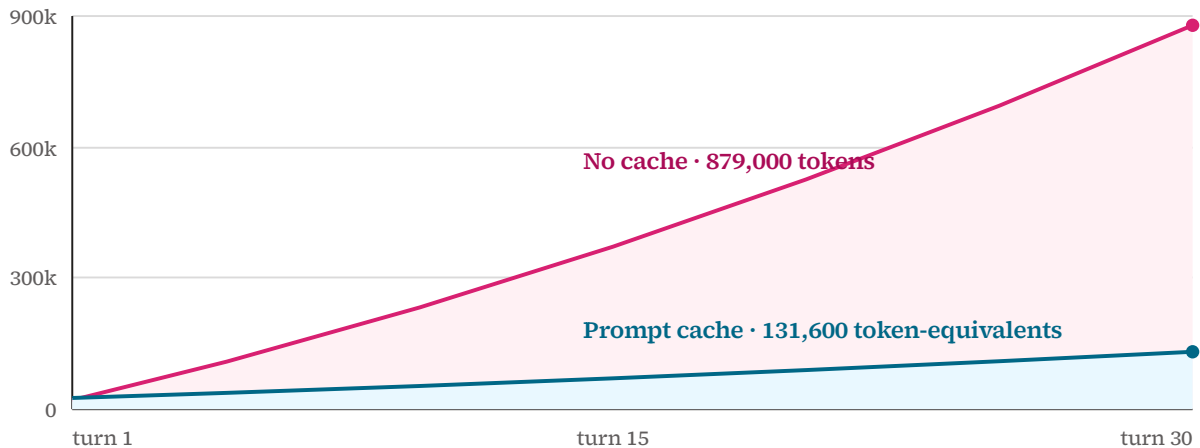
$$\begin{aligned}\text{cost}_1 &= w(n_0 + \Delta)p \\ \text{cost}_{t \geq 2} &= [r(n_0 + (t-1)\Delta) + w\Delta]p \\ \text{cost}_{\text{total}} &= \left[wn_0 + Kw\Delta + r((K-1)n_0 + \frac{1}{2}\Delta) \right]p\end{aligned}$$

The quadratic term — the growing history every turn re-reads — is billed at r . That is the whole point for agents.

EXHIBIT 5

Cumulative billed input tokens over a 30-turn episode: the quadratic term, at one tenth the rate

CUMULATIVE BILLED INPUT TOKENS



85% less input spend. At an illustrative \$3 per million input tokens: \$2.64 → \$0.39 per 30-turn episode.

$n_0 = 20,000 \cdot \Delta = 600 \cdot K = 30 \cdot w = 1.25 \cdot r = 0.10$. Output tokens excluded; they are unaffected by this layer.

The provider skips prefill exactly as the engine does, so TTFT on a hit is dominated by suffix prefill plus network — provider documentation reports reductions on the order of 80% for long cached prompts. The *first* request after a TTL expiry pays the write premium *and* full prefill latency; bursty traffic shows this as periodic latency spikes.

CODE

LISTING 5 · DECLARING A BOUNDARY AND READING THE ACCOUNTING BACK

```
import anthropic

client = anthropic.Anthropic()

def ask(question: str, shared_context: str, tools: list) -> str:
    resp = client.messages.create(
        model=MODEL,
        max_tokens=512,
        tools=tools,
        system=[{
            "type": "text",
            "text": shared_context,
            "cache_control": {"type": "ephemeral"}, # prefix ends here
        }],
        messages=[{"role": "user", "content": question}],
    )
    u = resp.usage
    print(f"uncached in={u.input_tokens}  cache_write={u.cache_creation_input_tokens}  "
          f"cache_read={u.cache_read_input_tokens}")
    return resp.content[0].text
```

LISTING 6 · A COST MODEL MATCHING THE FORMULA, TO SIZE THE BENEFIT FOR YOUR OWN WORKLOAD

```
def agent_loop_cost(n0: int, delta: int, turns: int, p: float,
                    w: float = 1.25, r: float = 0.10) -> tuple[float, float]:
    """(cached_cost, uncached_cost) for a loop that appends `delta` tokens per turn.
    Turn 1 writes the whole prompt; later turns read history and write only delta."""
    cached = w * (n0 + delta) * p
    cached += sum((r * (n0 + (t - 1) * delta) + w * delta) * p
                  for t in range(2, turns + 1))
    plain = sum((n0 + t * delta) * p for t in range(1, turns + 1))
    return cached, plain

c, u = agent_loop_cost(n0=20_000, delta=600, turns=30, p=3e-6)
print(f"cached ${c:.2f} vs uncached ${u:.2f} → {100*(1-c/u):.0f}% saved")
# cached $0.39 vs uncached $2.64 → 85% saved
```

LAYER 4

Semantic caching — in front of the model, matched by meaning

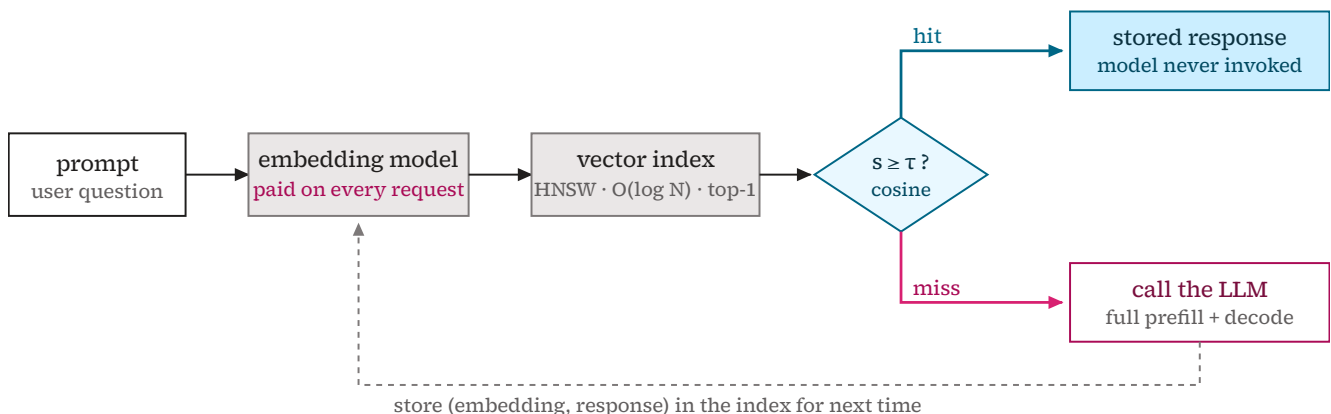
Owner: the application. Lifetime: a policy TTL. Saves: the entire LLM call — and it is the one layer that can be wrong.

ARCHITECTURE

The first three caches match on exact token sequences and reuse *intermediate computation*. Semantic caching is a different class of system: it sits in the application layer and reuses *final responses*. Its components are an embedding model (10–100 ms per API call, or ~1 ms for a small local model), an approximate-nearest-neighbour index, a response store with a TTL, and a similarity threshold τ . Production systems namespace the index by tenant or user, invalidate on knowledge updates, and often put a cheap exact-match layer in front. GPTCache and the semantic-cache modules in Redis and LangChain are reference implementations.

EXHIBIT 6

The one lossy layer: a threshold decides whether the model is consulted at all



A hit removes the entire LLM call — for a 500-token answer at ~40 ms per token that is 20 s of decode and all of the output-token cost. The savings profile is inverted relative to the other three: they trim prefill and input cost; this one trims decode and output cost.

MATHS

$$\begin{aligned}s(u, v) &= \frac{u \cdot v}{\|u\| \|v\|} & \text{hit} &\iff m \\ \mathbb{E}[\text{cost}] &= c_{\text{emb}} + (1 - h) (c_{\text{in}} + c_{\text{out}}) \\ \mathbb{E}[\text{latency}] &= l_{\text{emb}} + (1 - h) l_{\text{llm}} \\ \text{pays off when} & \quad h > \frac{c_{\text{emb}}}{c_{\text{in}} + c_{\text{out}}} \quad \text{and} \quad h > \frac{l_{\text{emb}}}{l_{\text{llm}}}\end{aligned}$$

Because output tokens cost several times more than input tokens and decode dominates l_{llm} , the cost break-even hit rate is usually a few percent. But **every miss still pays c_{emb} and l_{emb}** ; at $h \approx 0$ the system is strictly worse than no cache.

CODE

CORRECTNESS IS A CLASSIFICATION PROBLEM

Raising τ lowers the false-positive rate (a wrong answer served) and raises the false-negative rate (a missed hit). The trade-off must be measured per workload, because embedding distance is a poor proxy for answer equivalence exactly where it matters: **negation, numbers, entities and dates** can flip the required answer while barely moving the embedding.

RUNNING EXAMPLE, THIS LAYER

An agent's prompts are unique by construction — each contains fresh tool output — so $h \approx 0$ and a semantic cache would add embedding cost on every turn for nothing. Semantic caching belongs in front of *user-facing* question answering with heavy repetition, not inside an agent loop.

LISTING 7 · A COMPLETE IN-MEMORY SEMANTIC CACHE

```

import time
import numpy as np

class SemanticCache:
    def __init__(self, embed, threshold: float = 0.92, ttl_s: int = 3600):
        self.embed, self.tau, self.ttl = embed, threshold, ttl_s
        self.vecs: np.ndarray | None = None          # [N, d], rows L2-normalised
        self.items: list[tuple[str, str, float]] = [] # (prompt, response, inserted_at)

    @staticmethod
    def _norm(v: np.ndarray) -> np.ndarray:
        return v / np.linalg.norm(v)

    def lookup(self, prompt: str):
        q = self._norm(self.embed(prompt))          # paid on every request, hit or miss
        if self.vecs is None:
            return None, q
        sims = self.vecs @ q                        # cosine similarity via dot product
        i = int(sims.argmax())
        fresh = time.time() - self.items[i][2] < self.ttl
        hit = self.items[i][1] if (sims[i] >= self.tau and fresh) else None
        return hit, q

    def insert(self, q: np.ndarray, prompt: str, response: str) -> None:
        self.vecs = q[None, :] if self.vecs is None else np.vstack([self.vecs, q])
        self.items.append((prompt, response, time.time()))

def answer(cache: SemanticCache, llm, prompt: str) -> str:
    hit, q = cache.lookup(prompt)
    if hit is not None:
        return hit                                # model never invoked
    response = llm(prompt)
    cache.insert(q, prompt, response)
    return response

```

Swap the NumPy dot product for FAISS or an HNSW index beyond $\sim 10^5$ entries, and add eviction of expired rows for long-running processes.

LISTING 8 · MEASURING THE THRESHOLD RATHER THAN GUESSING IT

```
def sweep_threshold(embed, pairs, taus):
    """pairs: [(prompt_a, prompt_b, same_answer: bool)] labelled by hand."""
    norm = lambda v: v / np.linalg.norm(v)
    sims = np.array([float(norm(embed(a)) @ norm(embed(b))) for a, b, _ in pairs])
    same = np.array([s for _, _, s in pairs], dtype=bool)
    for tau in taus:
        hit = sims >= tau
        tp, fp, fn = (hit & same).sum(), (hit & ~same).sum(), (~hit & same).sum()
        prec = tp / (tp + fp) if tp + fp else 1.0
        rec = tp / (tp + fn) if tp + fn else 0.0
        print(f"tau={tau:.2f} precision={prec:.2f} recall={rec:.2f}")
```

Seed the labelled pairs with the hard cases: negations ("is X safe" / "is X unsafe"), swapped numbers, swapped entities, and different dates.

SYNTHESIS

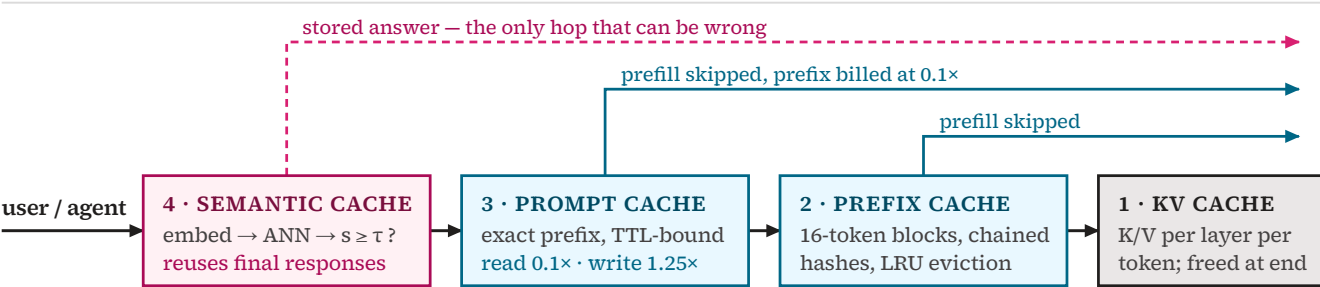
The four layers together

A request descends the stack from the application to the GPU, and any layer that hits short-circuits everything below it. Read Exhibit 7 left to right as the path of a single request; the band underneath records who owns each station and what it guarantees.

EXHIBIT 7

The request path: four caches, four owners, one lossy hop

EARLY EXIT — RESPONSE RETURNED WITHOUT REACHING THE GPU



On a miss the request passes to the next layer down; only the KV cache is unconditional.

OWNER	application	API provider	serving engine	model runtime
MATCHES ON	meaning	exact tokens	exact tokens	one request
LOSSLESS	No	Yes	Yes	Yes

Per 30-turn episode: the KV cache makes decode linear; prefix or prompt caching cuts prefill from ~7 s to ~0.2 s and input spend by 85%. Three layers change nothing about the output; the fourth would if it ever fired.

Layers 2 and 3 are the same mechanism seen from two sides of a commercial boundary: one is measured in FLOPs, the other in dollars.

SIDE BY SIDE

	KV CACHE	PREFIX CACHING	PROMPT CACHING	SEMANTIC CACHING
Owner	Model runtime	Serving engine	API provider	Application
Stores	K/V tensors, one request	K/V blocks, hash-keyed	K/V state (opaque)	Embeddings + responses
Match	Same request	Exact token prefix, block-aligned	Exact token prefix	Cosine similarity $\geq \tau$
Saves	$O(T^2) \rightarrow O(T)$ decode work	Prefill FLOPs, TTFT	Input cost ($\rightarrow$ 10%), TTFT	Input + output cost, full LLM latency
Overhead	Memory: $2 \cdot L \cdot n_{kv} \cdot d_h \cdot b$ per token	Hash lookup, cache memory	Write premium (1.25×)	Embedding call on every request
Lossless	Yes	Yes	Yes	No

Which cache do I need?

SITUATION	REACH FOR
Running your own inference	KV cache is automatic; enable prefix caching; order prompts stable→variable
Calling a provider API in a loop	Prompt caching with a breakpoint after the stable prefix; verify <code>cache_read_input_tokens</code>
Many users asking near-identical questions	Semantic cache in front, with a measured τ and hard-case tests
Agent loops with unique tool output	Prefix/prompt caching only; do not add a semantic cache
Prompt shorter than the provider minimum	Nothing to cache; restructure or accept full price

Common misconceptions

“Prompt caching and semantic caching are the same thing.” No: one is exact-prefix reuse of attention state and cannot change output; the other is similarity retrieval of stored answers and can.

“Caching only helps at temperature 0.” The three token-level caches reuse K/V state, which is identical regardless of sampling; the output distribution is unaffected.

“A cache hit means the model saw a shorter prompt.” The model still attends over the full context; only the *computation* of the prefix's K/V is skipped.

“Putting the user question first makes the model focus on it.” It also invalidates every cache block after it. Put stable content first.

“Prefix caching saves decode time.” It saves prefill. Decode cost per token is unchanged.

Check your understanding

- A model config lists `num_hidden_layers=32`, `num_key_value_heads=8`, `head_dim=128`. How much KV memory per token in bf16, and for a 32k context? $2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 131 \text{ KB/token}$; $32\text{k tokens} \approx 4.3 \text{ GB}$.
- Your prompt begins with "Request-ID: 8f3a...". What is the prefix-cache hit rate across requests? **Zero** — block 1's hash differs every time, and every later hash chains from it.
- With $w = 1.25$, $\tau = 0.10$, a prefix reused 4 times within the TTL costs what fraction of uncached? $(1.25 + 3 \cdot 0.1)/4 \approx 39\%$.
- A semantic cache has $c_{\text{emb}} = 0.5\%$ of an LLM call's cost. What hit rate must it exceed to save money? **0.5%** — and it must also clear the latency threshold $l_{\text{emb}} / l_{\text{llm}}$.
- Which of the four caches can return a wrong answer, and why? **Only semantic caching** — it matches on embedding similarity, not token identity.

Further reading

- Kwon et al., 2023 — *Efficient Memory Management for Large Language Model Serving with PagedAttention* (vLLM; block tables, sharing).
- Zheng et al., 2023 — *SGLang: Efficient Execution of Structured Language Model Programs* (RadixAttention).
- Ainslie et al., 2023 — *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*.
- vLLM documentation, “Automatic Prefix Caching” — block hashing and eviction details.
- Anthropic documentation, “Prompt caching” — breakpoints, TTLs, minimums, current pricing.
- GPTCache project documentation — semantic cache architecture and evaluation.